Al fin(ito) y más allá del bearer token

Un billete al portador llamado token



Un billete al portador: quien lo tiene, entra

- **Posesión = autorización.** No se verifica la identidad del portador.
- **«Bearer» viene del derecho mercantil:** instrumento al portador.
- **Si se pierde el billete,** quien lo levanta lo gasta.
- **Si se filtra el token,** quien lo tiene lo usa.

Fuente: RFC 6750, Bearer Token Usage (octubre 2012).

La norma ya lo admitía desde 2012

- **RFC 6749 §10.3:** el recurso no puede confirmar que el token se emitió a quien lo presenta.
- **La firma prueba el emisor,** no el destinatario.
- **El riesgo es de diseño,** no de implementación.
- **El bearer no falló:** cumplió exactamente lo que prometió.

Fuente: RFC 6749 §10.3 (octubre 2012).

Alrededor del token hay un sistema



Fuente: OWASP API Security Top 10 (2023).

No todas las APIs son REST

- **REST — recursos.** Riesgo: BOLA / IDOR.
- **GraphQL — un endpoint.** Consultas profundas y sobre-exposición.
- **gRPC — binario.** Los WAF no lo inspeccionan fácil.
- **SOAP — XML.** Arrastra XXE.
- **WebSocket — canal.** Suele autenticar solo al inicio.
- **Webhooks y eventos — push.** SSRF y replay si no se firman.

Fuente: OWASP API Security Top 10 (2023).

De la contraseña a la cookie

- **HTTP sin estado:** hacía falta recordar quién sos.
- **Basic:** usuario y contraseña en Base64 en cada request.
- **Digest:** un hash con nonce; mejora el secreto, no el modelo.
- **Cookie (Netscape, 1996):** un identificador de sesión; el estado queda en el servidor.

Fuente: RFC 2617 (1999) / RFC 7616 (2015); RFC 6265 (2011).

Firmar era seguro; ganó lo simple

- **WS-Security (2004) y SAML 2.0 (2005):** aserciones XML firmadas.
- **OAuth 1.0 (RFC 5849, 2010):** firmaba cada request con HMAC-SHA1.
- **Robar una petición no servía.**
- **Pero canonicalizar y repartir secretos** era una pesadilla de integración.

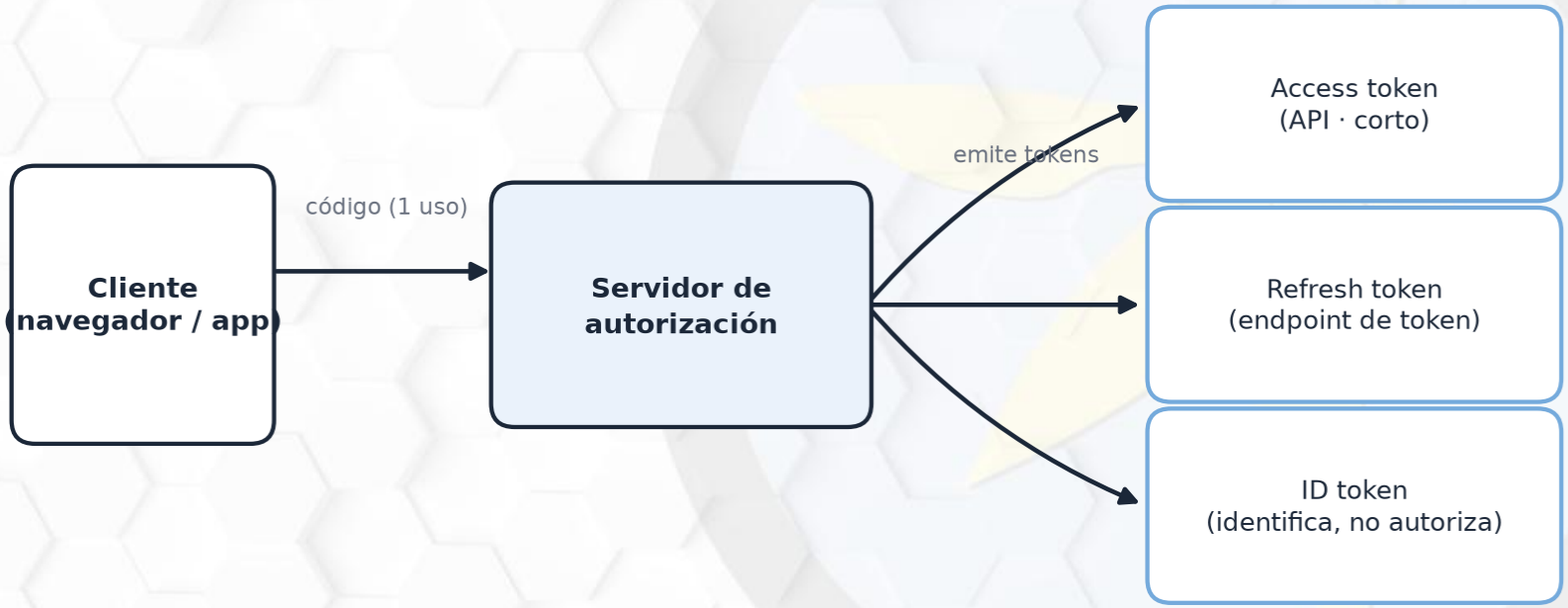
Fuente: SAML 2.0 (2005); WS-Security (2004); RFC 5849 (2010).

OAuth 2.0 nació con dos tipos de token

- **RFC 6749 (2012)** definía dos tipos: bearer y mac.
- **El tipo mac** firmaba componentes de cada request.
- **El borrador MAC se abandonó** en febrero de 2012.
- **Ganó la simplicidad:** con ella llegó RFC 6750.

Fuente: RFC 6749 §7.1 (2012); borrador MAC (febrero 2012).

No hay «un token»: hay una familia



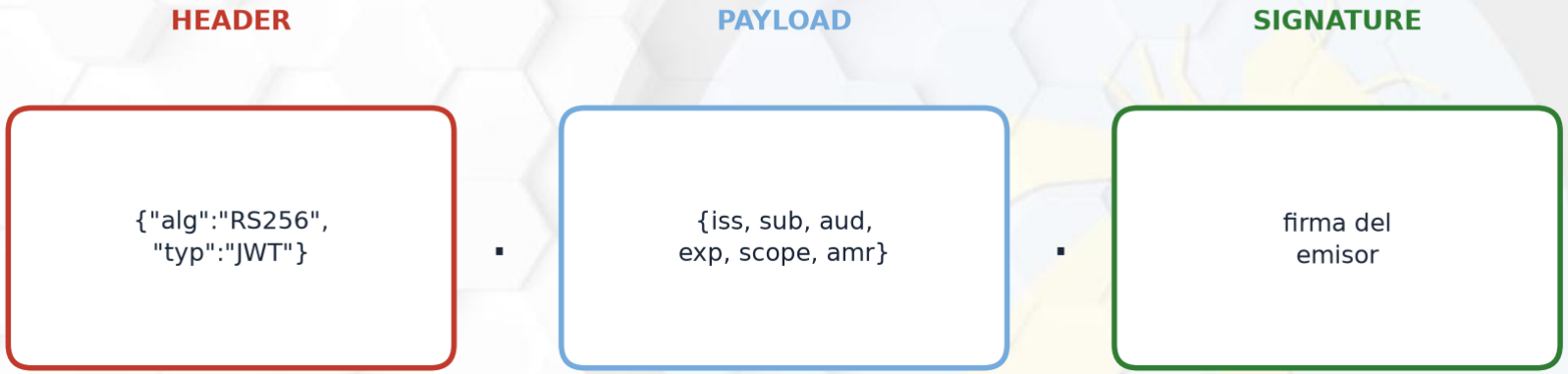
Fuente: RFC 6749 §4.1 y §5 (2012); OIDC Core (2014).

Cada token tiene destino y vida

Token	Para qué	Vida	Dónde viaja
Authorization code	Probar la autorización	Un solo uso	Navegador (front channel)
Access token	Llamar a la API	Minutos u horas	Cabecera, hacia la API
Refresh token	Renovar sin reautenticar	Días o semanas	Solo el endpoint de token
ID token	Decir quién y cómo	Corto	App cliente (no la API)

Fuente: RFC 6749 §1.5, §4.1, §5, §6 (2012); RFC 7009 (2013).

Firmado, no sellado



cada parte va en base64url · base64 NO es cifrado

firmado ≠ sellado: el payload lo lee cualquiera → JWE para confidencialidad

Fuente: RFC 7519 (2015); RFC 8725 (2020).

Cuatro preguntas que valida un JWT

- **¿Quién lo emitió?** — iss
- **¿Es para mí?** — aud
- **¿Está vigente?** — exp / nbf
- **¿Qué permiso tiene?** — scope
- **¿Cómo se autenticó?** — amr / acr

Fuente: RFC 7519 (2015); RFC 8176 (2017); RFC 9068 (2021).

El token en la URL quedó prohibido



Fuente: RFC 6750 §2 (2012); RFC 9700 (enero 2025).

La MFA protege un instante, no 30 días



Fuente: NIST SP 800-63-4 (julio 2025); OIDC Core (2014).

Passkeys sobreviven al phishing

- **NIST AAL1–AAL3:** la garantía se liga al riesgo.
- **AAL2:** dos factores de tipos distintos.
- **AAL3:** prueba criptográfica de una clave en hardware.
- **WebAuthn / passkeys (W3C, 2019):** el desafío va atado al origen.
- **TOTP se phishea en tiempo real;** SMS es el eslabón más débil.

Fuente: NIST SP 800-63-4 (julio 2025); WebAuthn (W3C 2019).

Tres ataques que la MFA no cubre

- **Fatiga de push (MITRE T1621):** el atacante ya tiene la contraseña y dispara pushes.
- **Phishing AiTM:** intercepta la sesión completa que ya pasó la MFA.
- **Infostealers y pass-the-cookie:** roban y reutilizan el artefacto.
- **En los tres, la MFA hizo su trabajo;** lo que falla es la posesión.

Fuente: MITRE ATT&CK T1621 · T1539; casos públicos.

A una máquina no se le pide un 2FA

- **El invocador** es un microservicio, un job de CI o un dispositivo.
- **No hay usuario:** no hay segundo factor que mostrar.
- **La pregunta correcta:** ¿qué workload habla y con qué credencial?
- **Mecanismos:** client credentials, private_key_jwt, mTLS, DPoP.
- **El MFA del sistema** es una identidad de vida corta y verificable.

Fuente: RFC 6749 §4.4 (2012); RFC 7523 (2015); RFC 8705 (2020).

Persona y sistema: dos preguntas

	Persona	Sistema (M2M)
Qué se prueba	Que un humano se autenticó	Que un workload pidió el token
Segundo factor	Passkey / OTP / push	No existe
Artefacto	Sesión + access token corto	Token corto o credencial efímera
Vida típica	Minutos u horas	Muy corta, rotada
Fallo típico	Fatiga, AiTM, robo de sesión	Secretos eternos en repos y CI
Revocación	Cerrar sesión / revocar	Rotar la credencial

Fuente: NIST SP 800-63-4 (2025); GitGuardian (2026).

El token es una puerta, no el edificio

Api1 · BOLA — autorización a nivel de objeto

Api2 · Autenticación rota

Api3 · BOPLA — exposición de propiedades

Api4 · Consumo de recursos sin límites

Api5 · BFLA — autorización a nivel de función

Api6 · Acceso a flujos de negocio sensibles

Api7 · SSRF

Api8 · Configuración de seguridad incorrecta

Api9 · Gestión impropia del inventario

Api10 · Consumo inseguro de APIs de terceros

El token es la puerta; estos riesgos viven en todo el edificio.

Fuente: OWASP API Security Top 10, edición 2023.

El edificio tiene más habitaciones

- **Autorización a nivel de objeto.** Que puedas ver el pedido 5678 y no solo el 1234.
- **Validación de entrada:** inyección, SSRF, deserialización, mass assignment.
- **Gateway y tráfico:** cuotas, rate limiting y gestión de bots.
- **Inventario:** versiones viejas y APIs fantasma que siguen vivas.
- **Cadena de suministro:** secretos en CI y dependencias de terceros.
- **Privacidad y cumplimiento:** Ley 18.331, Decreto 66/025 y reporte a CERTuy.

Fuente: OWASP API Top 10 (2023); NIST; AGESIC / CERTuy.

Se emitió mal, no se revocó, se filtró

- **Facebook «View As» (2018):** token emitido al usuario equivocado.
- **Okta / Cloudflare (2023):** una sesión robada que siguió sirviendo.
- **Codecov (2021):** tokens del CI exportados por un script modificado.
- **En los tres,** el token funcionó como fue diseñado.

Fuente: Meta (2018); Cloudflare (2023); Codecov (2021).

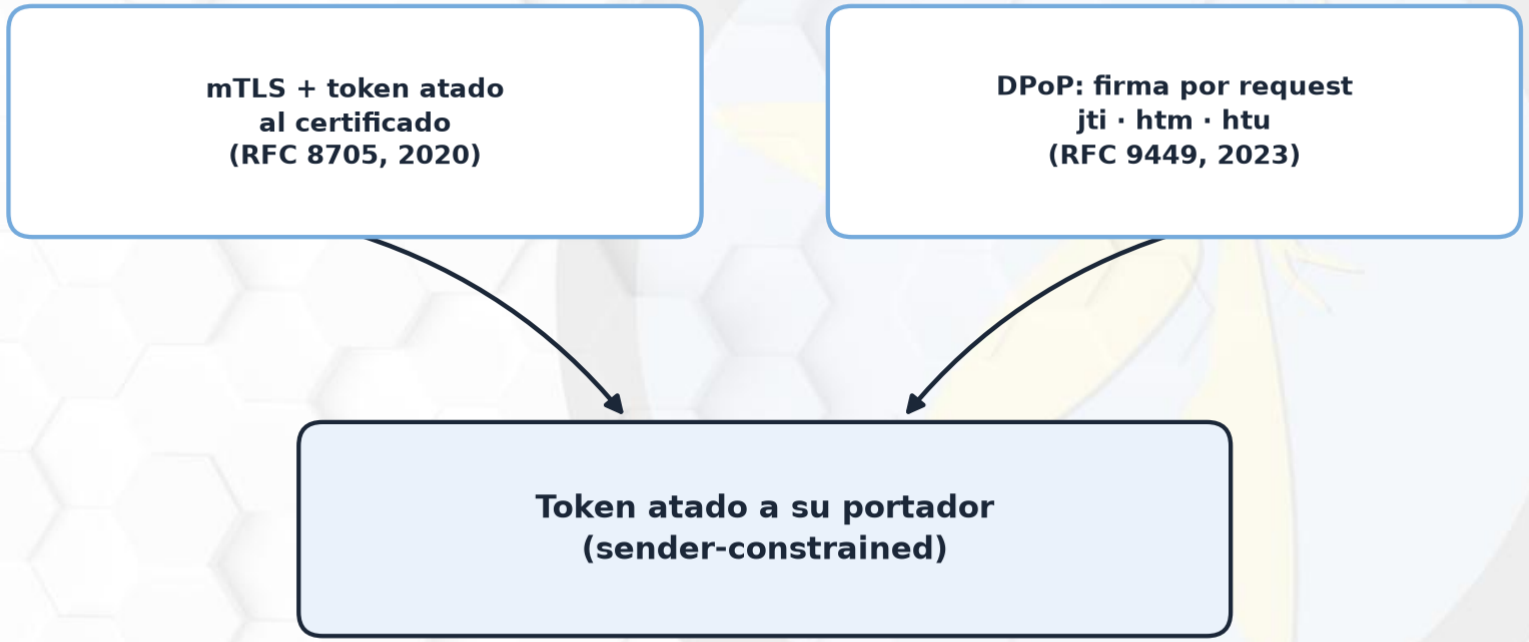
Después de aceptarlo, queda trabajo

- **Uber (2022):** la MFA no sobrevive al robo de sesión.
- **BOLA:** token válido, objeto ajeno (~40 % de los ataques a APIs).
- **JWT mal validado:** alg:none, confusión HS/RS, inyección de kid.
- **Ninguna falla es del string:** son de emisión, autorización o verificación.

Fuente: UpGuard / NYT (2022); OWASP (2023); RFC 8725 (2020).

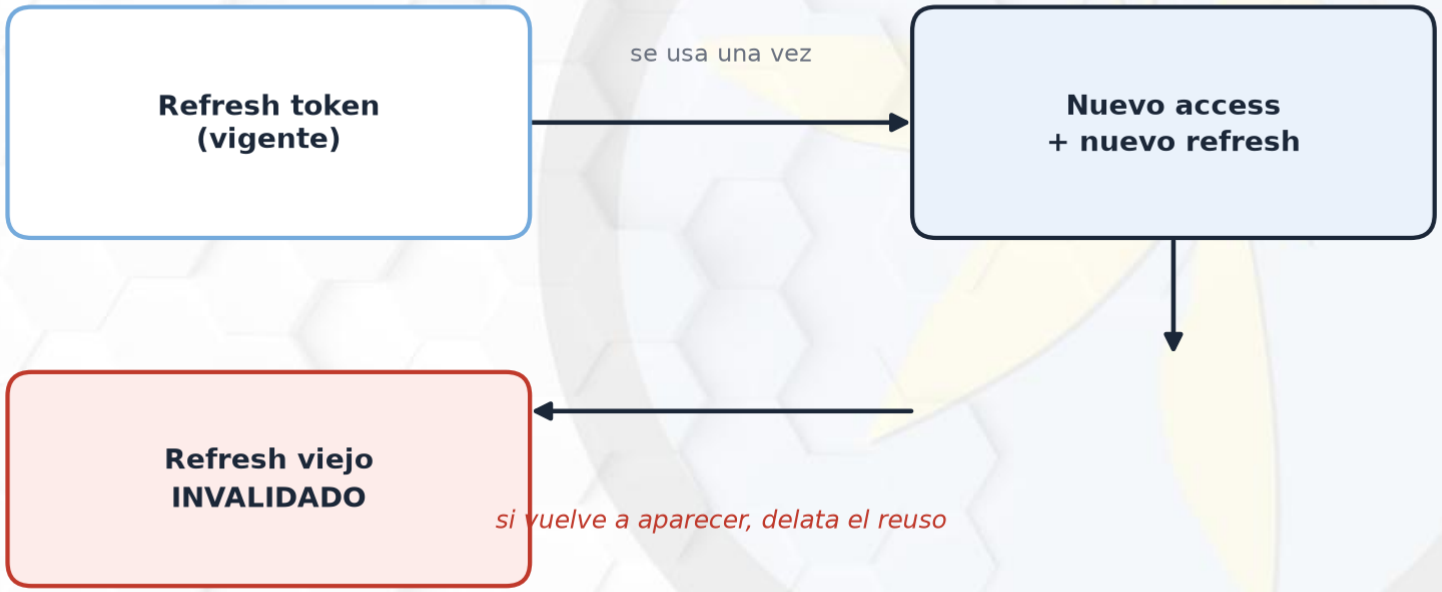
Atar el token a su portador

Si la posesión no basta, se exige prueba de posesión.



Fuente: RFC 8705 (2020); RFC 9449 (2023); RFC 9700 (2025).

Rotar el refresh lo vuelve de un solo uso



Fuente: RFC 6749 §10.4 (2012); RFC 7009 (2013); RFC 9700 (2025).

Vida corta, rotación y revocación

- **Access token:** vida corta y audiencia estricta.
- **Refresh token:** rotación y revocación.
- **Permisos:** mínimo privilegio en los scopes.
- **Detección de anomalías:** ¿este token se usa desde dos países a la vez?
- **Claves de máquina:** secret manager y rotación automática.

Fuente: RFC 9700 (2025); RFC 7009 (2013); GitGuardian (2026).

Ocho preguntas antes de confiar

- ¿El algoritmo de verificación sale de la configuración?
- ¿Validás iss, aud y exp con tolerancia corta?
- ¿El access token vive lo que debe?
- ¿Hay rotación y revocación de refresh?
- ¿La autorización por objeto está resuelta?
- ¿El token viaja en cabecera y nunca en la URL?
- ¿La identidad de máquina es de vida corta?
- ¿Qué queda en los logs?

Fuente: RFC 8725 (2020); RFC 9700 (2025); OWASP API Top 10 (2023).

Un dominio, un mensaje

Dominio	La pregunta que importa	Dónde está
Identidad	¿El invocador es persona o sistema?	RFC 9700; NIST
Autorización	¿Esta identidad puede tocar este objeto?	OWASP Api1 / Api5
Entrada	«No confío»: validá todo	OWASP Api3 / Api8
Tráfico	¿Hay límites y control de abuso?	OWASP Api4; gateway
Observabilidad	¿Se ve para poder revocar?	RFC 7009 / RFC 7662
Cumplimiento	¿Qué exige la norma?	Ley 18.331; CERTuy

Fuente: OWASP API Top 10 (2023); NIST SP 800-63-4 (2025); RFC 9700 (2025).

El token se acaba; el riesgo no

- **El token es finito:** emitir, rotar, revocar.
- **La MFA protege la emisión,** no la posesión.
- **Autenticar es la puerta;** autorizar es la cerradura.
- **La seguridad nunca estuvo en el string:** está en el sistema.

Fuente: Bibliografía consolidada al final de la clase completa.